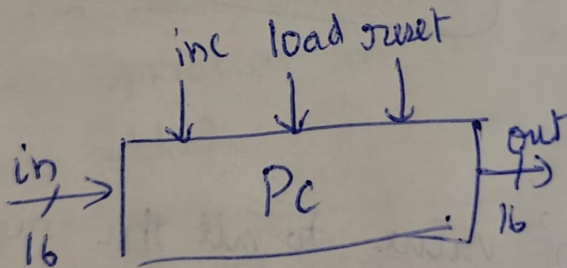


Program Counter



* It is a priority logic.

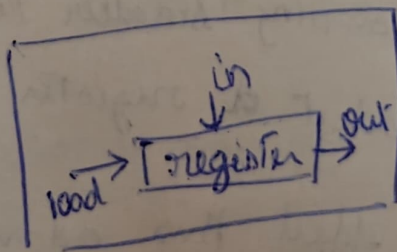
if reset(t) : $out[t+1] = 0$
else if load(t) : $out[t+1] = in(t)$
else if inc(t) : $out[t+1] = out[t] + 1$
else $out[t+1] = out[t]$



Logically

→ reset has highest priority

because when reset == 0, irrespective of 'load' and 'inc', my Register inside PC must be 0



↳ * needed to store the value and perform an operation (reset/inc/load)

* Something has to remember present state

If $reset = 1$, $out = 0$

else

$load = 1$, $out = in$

else

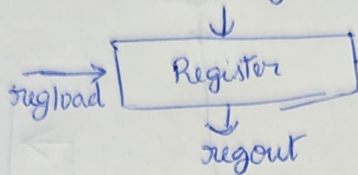
$inc = 1$, $out = present\ state + 1$

else

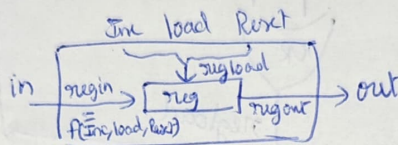
$out = present\ state$

Priority $reset \uparrow \gg load \gg inc$

* Consider the register in the PC



Whatever changes I make to my present state, I must save it in this register.

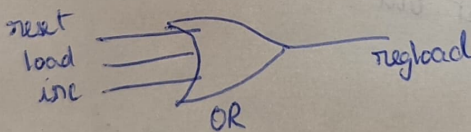


and if I am not performing any operation, the present state must be outputted as it is

$\Rightarrow reset = 0 \ \& \ inc = 0 \ \& \ load = 0$

If any one $\rightarrow$ OP gets modified.

$\hookrightarrow$ can be realised using OR gate.



According to priority, $reset \gg load \gg inc$

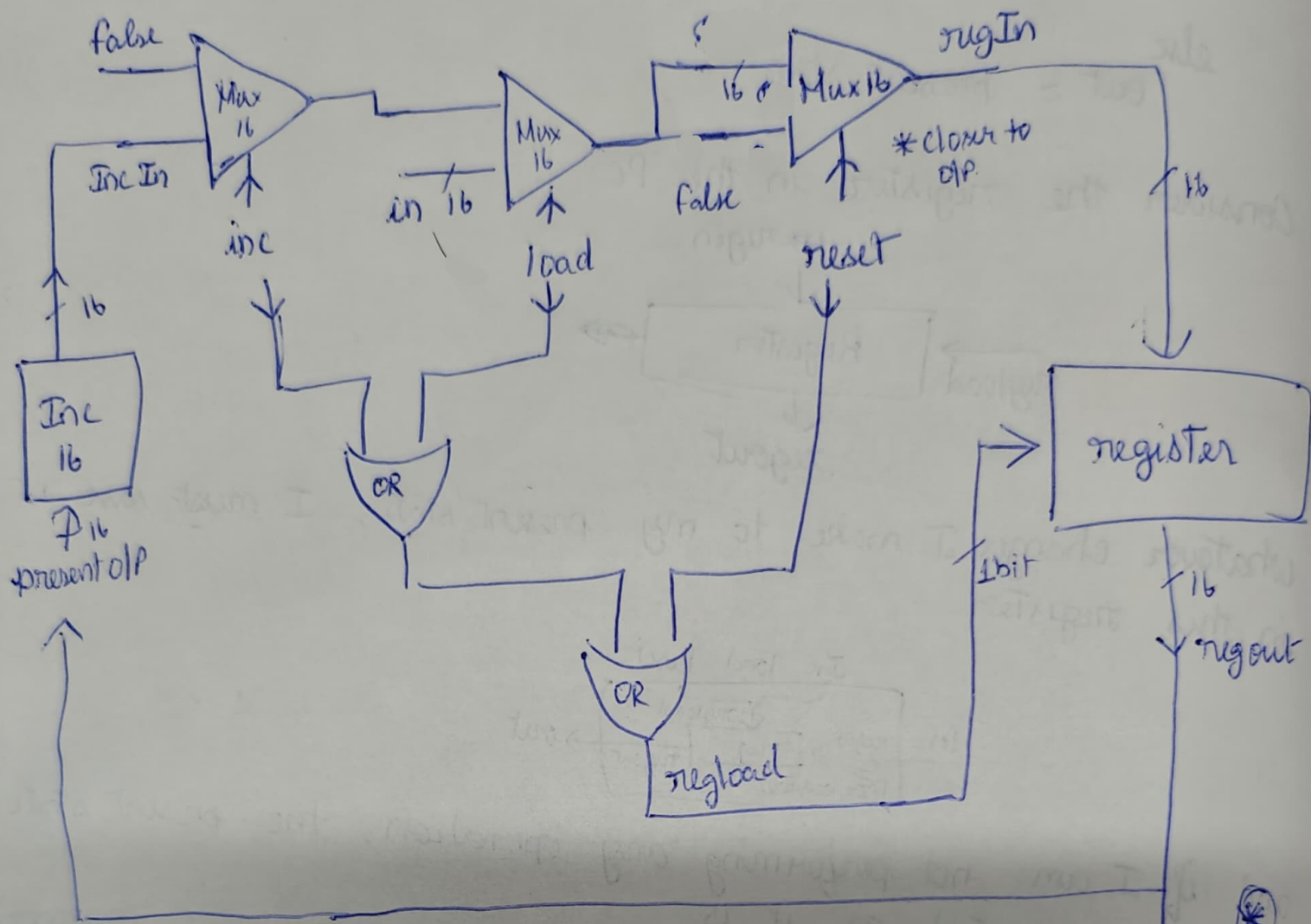
TT

Inc	load	Reset	Output
0	0	0	present state
x	x	1	0000 / all 0's
x	1	0	load in [t]
1	0	0	present state + 1

* when designing $\rightarrow$ highest priority $\rightarrow$ closer to O/P



Full Design of a Program Counter



In this HDL, out_{port} cannot be directly used as an input

* So we are making regent as intermediate part and used 2 Inv to get out.

